Question Answering Over Knowledge Graphs: The DBpedia Case Study

Ilie Cristian Dorobăț
Department of Computer Science
National University of Science and Technology POLITEHNICA Bucharest
Bucharest, Romania
ilie.dorobat@upb.ro

***Abstract*—In contrast to relational database management systems, which require numerous joins and intricate queries to handle complex interconnected data, graph databases excel precisely due to their storage approach which involves keeping the data as nodes and edges, effectively representing entities and the relationships that connect them. However, querying graph databases can be perceived as a cumbersome process, especially by the non-technical audience, since it requires not only an understanding of data structures but also the technical specifications of the used protocol, usually SPARQL. To overcome this challenge, Question Answering over Linked Data systems provides users with the ability to query datastores using natural language. The current work presents a new architecture of the onIQ system, which, in addition to enhancing the syntactic parsing mechanism brings users the ability to easily enrich their statistical model with new named entities. The results of evaluating the system against the QALD-8 test dataset are promising, showing the highest precision parameter value among the analyzed systems.**

Keywords—Natural Language Interface, Knowledge Discovery, Question Answering, SPARQL

I. Motivation

The RDF Triplestore [1], hereafter referred to as triplestore, is a graph database whose knowledge base (KB) consists of a network of interconnected elements represented in the form of *subject-predicate-object* relationships, also termed as *triples* or *statements*. For example, the information *"Bucharest is in Romania"* is represented in DBpedia using the triple <dbr:Bucharest dbo:country dbr:Romania>. However, to answer to inquiries such as *"What are the 5 highest mountains in Romania?"*, it becomes necessary to construct a query adhering to the syntax of the protocol supported by the triplestore, with SPARQL being the most widely used protocol for this purpose.

While the triplestore's KB is frequently augmented with external terms and concepts, the act of querying these elements by regular users can prove to be arduous [2]. This is because, in addition to possessing minimal technical expertise, users must be familiar with both data representation techniques within the triplestore and the protocol's syntax. Question Answering over knowledge bases systems aim to solve this problem precisely by relieving users of the task of translating natural language questions into SPARQL queries. Basically, this process, which takes the shape of "semantic parsing" [3], can take various forms, but in principle it involves syntactic analysis of the query in order to identify named entities and predicates so that they can then be mapped to resources and properties in the target vocabulary, and finally a valid SPARQL query can be generated. Other solutions for transforming natural language queries into SPARQL queries are template-based, which are generally limited to previously chosen templates [4].

The current paper introduces the approach used to develop a natural language interface (NLI) for triplestores whose role is to understand the requests passed in natural language and to translate them into SPARQL queries. The first layer of the architecture shown in Fig. 1 is represented by the *configuration of the system* with named entities found in the KB and the extraction of all the OWL classes that will be used to determine the type of resource searched. The next layer is *Question Pre-processing*, which involves enriching the plain text with NLP-specific annotations that allow us to identify Part-of-Speech (POS), Dependencies (DEPs), named entities and other useful features in NLP. In the *Information Extraction* layer, the KB resources are identified, named entities are enriched with special tags and existing relationships between resources are extracted. The last layer involves *building triples, extracting filtering and ordering rules and finally building the SPARQL query*.

The evaluation was performed on the 8th Question Answering over Linked Data challenge (QALD-8) test dataset [5] because at the time of the development of the onIQ framework [6], this was a relatively new test dataset for which there were a number of tests conducted against it in the literature such that a comprehensive comparative analysis could be supported. Finally, the results are compared via GERBIL QA [7].

II. Related Work

The research literature provides a wide range of QA over Semantic Web solutions, with many of them leaning towards the utilization of templates rather than syntactic parsing [3]. gAnswer2 [8] focuses on addressing the challenge of querying RDF repositories from a graph data-driven perspective. It introduces a semantic query graph to model query intentions in a structural form, thereby simplifying the complexity of querying an RDF repository into a subgraph matching problem. QAnswer [9] is a QA system that prioritizes semantics over syntax that operates by generating a comprehensive range of potential queries based on the input natural language query and through an extensive training data, the system learns to discern the most suitable SPARQL query to use. qaSQP [10] is a system designed to create a query graph for a given natural language question using a structural query pattern. The concept driving this system is to recognize the outline or framework of the query graph inherent in the question, which is then enriched with query forms.

TeBaQA [11] is a QA system based on isomorphic graph patterns. The system's workflow commences by excluding semantically irrelevant terms to generate meaningful n-grams, and then a statistical classifier is used to detect questions sharing akin semantic structures. For the identification of named entities, relations, and classes, TeBaQA employs a KB-agnostic index-oriented search strategy. All entities in a

979-8-3315-5818-5/26/$31.00 ©2026 European Union

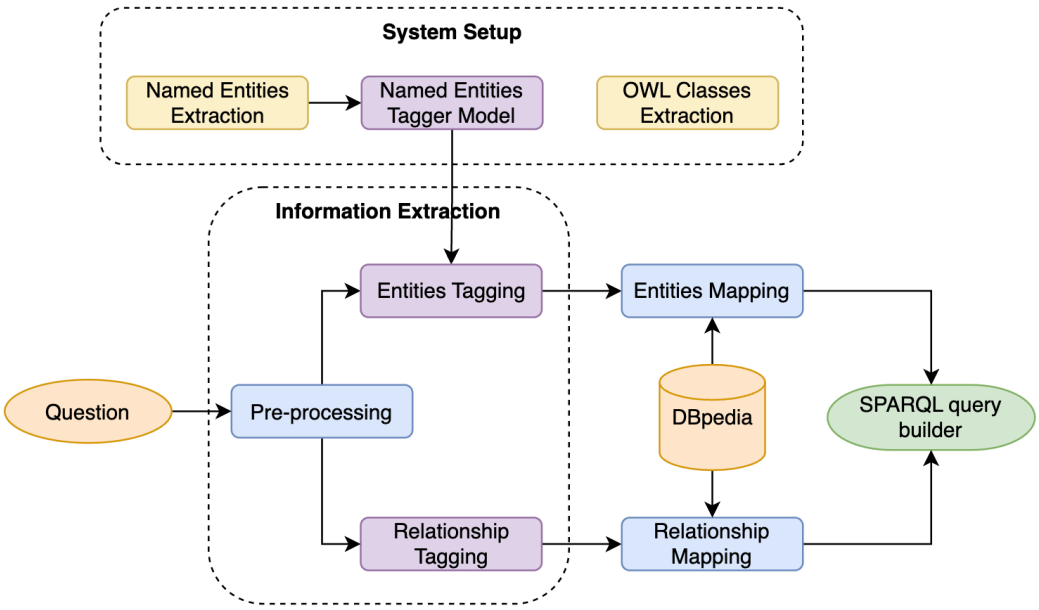


Fig. 1. The architecture of the proposed approach.

knowledge graph are embedded within an index, streamlining the correlation between pre-processed n-grams and named entities. OWL relations and classes are located in separate indexes, each serving the purpose of associating n-grams with relations and classes found in the knowledge graph ontology. However, if the labels contained in these three indexes exceed the token count of the target n-gram, a Levenshtein-distance filter of 0.8 is employed on the potential candidates. Ultimately, the templates are populated with the extracted details, and a selection is made based on a ranking mechanism to determine the most appropriate solution.

III. Preparation Stage

A. System Setup and Question Preparation

Configuration is the preliminary step to the whole process of transforming natural language questions into SPARQL queries which involves extracting all named entities and OWL classes from the KB. The list of named entities allows the model to be enriched with new named entities so that the parser can identify them in plain text, and the classes will be used at a later stage to determine the type of resource searched.

B. Question Pre-processing

The process of translating natural language queries into SPARQL queries starts with syntactic analysis. In particular, through spaCy [12], each part of the input is enriched with NLP-specific linguistic annotations used to determine POS, identify the dependency label between two words, recognize named entities and their type (people, organizations, geographical locations, events, time periods, art objects, etc.), extract the canonical form of each input component as well as other useful features in NLP.

POS tagging is an essential pre-processing task for a wide range of NLP goals [13] that involves annotating lexical terms with corresponding natural language parts of speech such as noun, verb, adjective, etc. To perform this task spaCy uses English statistical models trained on the OntoNotes 5.0 corpus [14], providing accuracy right up to the current state-of-the-art, but at the same time being the most powerful NLP parser [15]. For instance, Fig. 2 shows the result of POS tagging against the question *"What is the highest mountain in Romania?"* for which the spaCy tagger annotates the words with the following POS: personal pronoun (PRON); auxiliary verb (AUX); determiner (DET); adjective (ADJ); noun (NOUN); adposition (ADP); proper pronoun (PROPN); punctuation mark (PUNCT).



Fig. 2. POS tagging against the question *"What is the highest mountain in Romania?"*.

However, although the added annotations provide rich meaningful information about each individual word, they cannot provide information about the grammatical structure of the text under analysis [16].

Dependency parsing, as it is still known in research literature, is the step in which the relationships between terms in the text are examined in order to construct a dependency tree that provides the grammatical context of the text. For this, spaCy uses a variant of the non-monotonic arc-eager transition-system [17] together with a "break" transition to allow performance of text segmentation, and a pseudo-projective dependency transformation is used for non-projective parsing [18].

Continuing the example presented above, Fig. 3 shows the dependency tree generated for the analysed question. The dependency tree is basically a graph containing a set of nodes $\Omega = \{w_0, w_1, \ldots, w_n\}$ where w_0 is the root node and a set of dependency links $G = \{g_1, g_2, \ldots, g_n\}$ where g_i is an index in Ω representing the governor of w_i. In a different vein, the relationship $g_3 = 1$ indicates that w_1 is the governor for w_3. With the exception of the root node which has no governor, each node is in a dependency relationship with a single parent node (head) [19]. If we take a closer look at the generated dependency tree, we can see that the word *"is"*, which has the value of root node of the traversed query, is in a direct relationship with the words *"What"* and *"mountain"*, which we note as follows attr("is","What") respectively nsubj("is","mountain") where *attr* and *nsubj* are the types of

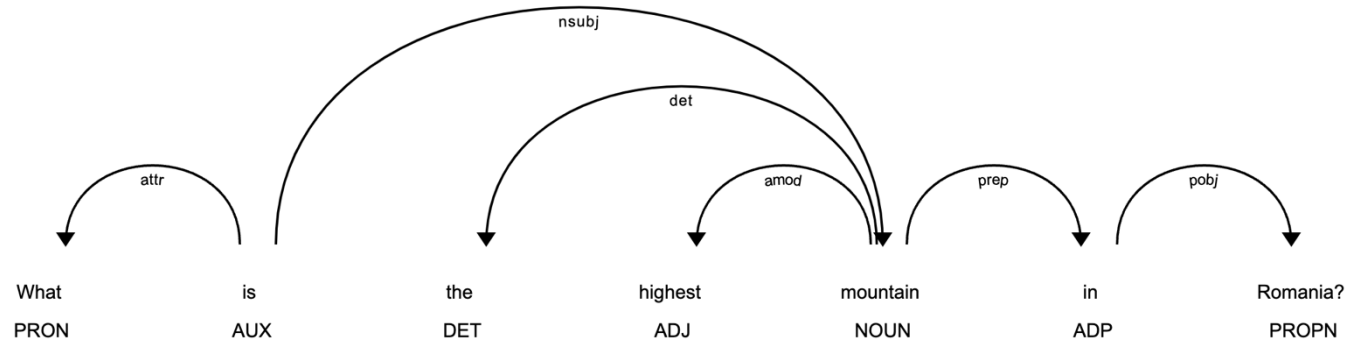


Fig. 3. Dependency tree generated for the question *"What is the highest mountain in Romania?"*.

relationship between each of the two pairs of words (attribute and nominal subject).

In addition, during this stage, spaCy constructs two more subgraphs for each node of the main graph, containing, if any, the words to the left and right of it that are in a direct dependency relationship. An eloquent example can also be seen from the dependency tree shown in Fig. 3. The node *"mountain"* is the governor for three other nodes, two of which are located to the left and one to the right of it, forming the following relationships on the left side: det("mountain","the") and amod("mountain","highest") and on the right hand side: prep("mountain","in") . Basically, through these subgraphs, the user can easily navigate through the descendants of a node.

After constructing these graphs, the next step is to identify and categorize key information (entities) in the text, also known as named entity recognition (NER), entity chunking, entity extraction or entity identification. This stage involves identifying the words that form named entities in an unstructured text into predefined categories such as names of persons and organisations, geographical locations, temporal expressions, events, units of measurement, monetary values and others. Finally, the spaCy pre-processing pipeline ends with the assignment of the basic form of each word by rules based on POS tags, or lookup tables, a process also known as lemmatization.

In the example illustrated, the only named entity identified is the word *"Romania"*, which is classified as a geographical area that is associated with a form of political structure such as a country, a region, a locality, etc. As far as lemmatization is concerned, the only differences can be observed in the case of the words *"is"* and *"highest"*, whose lemmatized form differs from the original value: *"be"* and *"high"*. For the rest of the words, the lemmatised form coincides with the values found in the analysed text, with the exception that if they are not recognised as named entities, the values will be lowering (e.g.: *"What"* will be lemmatised to the form *"what"*).

C. Identifying the Answer Type

After the spaCy pipeline is completed, the next step is to predict the expected answer type, which will be used later both for setting exceptions and for determining the result form of the SPARQL query.

Starting from the syntactic analysis performed above and based on a set of defined and applied heuristic rules, the question submitted by the user is classified into one of the 10 question types proposed in Table 1. Subsequently, using the identified question type, we proceed to the actual prediction of the answer type expected by the user which in principle is provided by the very first word of a question expressed in English and is limited to the following types: i) boolean value (equivalent to "yes" and "no"); ii) numerical value (number of occurrences); iii) geographical location; iv) period of time; v) person; vi) thing; vii) list of items. In this context, a thing is anything else except geographic locations, persons, temporal expressions, numbers or boolean values, while an object is everything else listed including the thing.

TABLE I. LIST OF QUESTION TYPES

Question Type	Description/Example	Expected Answer Type
DECISION	The question starts with an auxiliary verb. E.g.: *"Did Arnold Schwarzenegger attend a university?"*	True/ False
S_NOUN	The question starts with a noun. E.g.: *"Desserts from which country contain fish?"*	Thing/List
S_PREP	The question starts with a preposition. E.g.: *"In which country is Mecca located?"*	Thing
S_VERB	The question starts with a verb (other than auxiliary verb). E.g.: *"Give me the currency of China."*	Thing/List
WHO	The question begins with one of the words *"who", "whom", "whose"*. E.g.: *"Who is the manager of Real Madrid?"*	Person
WHEN	The question starts with the word *"when"*. E.g.: *"When did the Ming dynasty dissolve?"*	Temporal
WHERE	The question starts with the word *"where"*. E.g.: *"Where was the person born whose successor was Le Hong Phong?"*	Location
COUNT	The question starts with the constructions *"how many", "how much"*. E.g.: *"How many ethnic groups live in Slovenia?"*	Number
HOW_ADJ	The question starts with the word *"how"* followed by an adjective. E.g.: *"How high is the Yokohama Marine Tower?"*	Number/ Thing
WH_OTHERS	The question starts with a WH-word other than those listed above. E.g.: *"What is the net income of Apple?"*	Thing

From Table 1 we can see that the expected answer for any question starting with an auxiliary verb is "yes" or "no", whereas for a question starting with words "who", "whom",

"whose" it will be a person. If the question starts with the word "where", it is clear that a geographic location is to be discovered, and if the starting word is "when", we expect the answer to be a calendar date, a year or some other temporal expression. For the other cases, the expected answer is a thing or a list of objects, except if the question starts with "how many" or "how much" for which the user's expected answer is a numeric value indicating the quantity of objects queried.

IV. Information Extraction

Any semantic fact stored in a triplestore is represented as triples of form `<subject predicate object>`, whose elements can take the shape of RDF resources (abstract entities identified by an URI), and objects can also be literals consisting of a data value and optionally a language tag [20]. Therefore, in order to construct these triples, it is necessary to identify the components of the triples according to the role they play, namely: named classes, entities, relations.

A. Identification of KB Resources

Identifying the classes in the KB is a necessary step that allows us to narrow down searches for specific resource types. To do this it is first necessary to build an index containing all OWL classes in the KB, and then the lemmatised form of the target n-gram is compared with the lemmatised form of the labels of each OWL class in the created index.

For instance, considering the following question: "*Which soccer players were born on Malta?"*, the word group *"soccer players"* is identified in the POS tagging step as a compound noun which tells us that the query should be targeted only to resources describing *"soccer players"*. Basically, this is the 2-gram target whose lemmatised form will be used to identify the resource describing that group of words. In this case, the resource dbo:SoccerPlayer[1] is identified, and then this will be used to narrow down searches in the triplestore.

B. Named Entity Annotation

DBpedia Spotlight [21] (hereafter referred to as Spotlight) is a tool that allows automatic annotation of DBpedia mentions in plain text. However, it may contain not only named entities, but also common names such as *"basketball"*, *"company"*, *"president"*, *"soccer"*, etc., which is why Spotlight does not perform well on datasets for which only named entity identification is required. For instance, in the case of the previously analysed question, Spotlight it will annotate both the word *"soccer"* and the named entity *"Malta"* with the corresponding DBpedia resources.

The solution to this challenge is to use an NLP library for NER such as spaCy, and then annotate them with DBpedia mentions via Spotlight [22]. In this way we will ensure that annotation will only be done to named entities spotted by a feature that uses a mathematical model trained for this purpose, which can also be configured and trained to identify named entities found in particular domains such as medicine, geology, legal, etc. Basically, in the above example, spaCy will identify the word *"Malta"* as a named entity, and Spotlight will associate the resource dbr:Malta to it.

C. Extracting Existing Relationships

Representation of relationships in a KB is achieved through the properties existing in the vocabulary, the best match being identified by applying the set of heuristic rules against the properties related to queried resources. The algorithm used to calculate the similarity of a given string against an RDF property receives two parameters which are represented by the string for which similarity is to be calculated and an RDF property against which similarity is calculated. A tokenizer method is used to break the string into words called tokens and to filter these tokens to exclude punctuation, spacing and stop words. If the list of tokens generated is empty, a similarity score of 0 will be returned. Otherwise, the similarity calculation shall be initiated.

Before the actual calculation of the similarity score, the score variable that stores the result of this calculation is initialized to 1.0 with the idea that the score is calculated as the product of the old score and the matching score of each token. The determination of the latter score is done by summing the cosine similarity calculated for the two words and a buffer whose value of 1.0 ensures that the score will not be distorted if the similarity vector is 0. Finally, the score is aggregated by extracting the root of order `len(tokens)`, where tokens is the list of tokens obtained after applying the tokenizer against the input string, and the value of the buffer is subtracted.

For instance, for the following question: *"How many companies were founded by the founder of Facebook?"*, the named entity dbr:Facebook and the resource type dbo:Company are identified. Therefore, even if the result of the query must be a list of companies (resource type dbo:Company), in order to increase the accuracy of the result it is necessary first to identify the most appropriate property that matches the context of the other resource involved, dbr:Facebook from the context: *"founder of Facebook"*, and only then to identify the most appropriate property that applies to the main context *"companies were founded"*.

In this case, the identification of the most suitable existing relationship for the context *"founder of Facebook"* is done by applying the set of heuristic rules described above on the set of properties related to named entity dbr:Facebook extracted by means of the query presented in Listing 1.

```
PREFIX dbr: <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-
ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?property ?label ?subclassOf ?domain
?range
WHERE {
  dbr:Facebook  ?property   ?founder .
  ?property     rdf:type    ?subclassOf .
  ?property     rdfs:label  ?label .
  OPTIONAL { ?property   rdfs:domain   ?domain } .
  OPTIONAL { ?property   rdfs:range    ?range } .
  FILTER (
    ?property NOT IN (
      rdf:type, rdfs:subPropertyOf, rdfs:subClassOf
    ) &&
    ?subclassOf = rdf:Property
  ) .
  FILTER (langMatches(lang(?label), "en"))
}
ORDER BY ?property
```

Listing 1. SPARQL query used to extract properties of the resource describing the context "*founder of Facebook"*.

[1] dbo stands for http://dbpedia.org/ontology/

Then the same set of heuristic rules is applied to identify the most suitable relationship that can be applied to the extended context *"companies were founded by the founder of Facebook"*. In Listing 2 it can be seen that the query used in the latter case basically restricts the result to the properties in the KB applied only to resources of type dbo:Company and whose objects identified by the triple `<?many_companies property ?founder>` coincide with the values of the objects identified by applying the dbp:founder relationship to the resource dbr:Facebook.

```
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbp: <http://dbpedia.org/property/>
PREFIX dbr: <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-
ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?property ?label ?subclassOf ?domain
?range
WHERE {
  ?many_companies  ?property   ?founder .
  ?many_companies  rdf:type    dbo:Company .
  dbr:Facebook     ?property   ?founder .
  ?property        rdf:type    ?subclassOf .
  ?property        rdfs:label  ?label .
  OPTIONAL { ?property   rdfs:domain   ?domain } .
  OPTIONAL { ?property   rdfs:range    ?range } .
  FILTER (
    ?property NOT IN (
      rdf:type, rdfs:subPropertyOf, rdfs:subClassOf
    ) &&
    ?subclassOf = rdf:Property
  ) .
  FILTER (langMatches(lang(?label), "en"))
}
ORDER BY ?property
```

Listing 2. SPARQL query used to extract resource properties describing the extended context *"companies were founded by the founder of Facebook"*.

V. Building the Formal SPARQL Query

Translating a question into a semantically equivalent SPARQL query is a more complex process than simply identifying entities, the relationships between them and classes. After identifying these elements, it is necessary to determine the need to use aggregate functions, solution modifiers and order modifiers. The current subsection is devoted to considering all the implications of constructing such a SPARQL query.

A. Raw Triples

Unlike most QA over Semantic Web systems that use query templates to design formal queries, onIQ relies only on syntactic parsing, i.e. traversing the entire dependency tree in order to identify each dependency relation of type dep(w_n, w_k) starting from the root node w_0 to the last node. Basically, the parser identifies relations of type nominal subject, adjectives and prepositions to use them to generate raw triples, where raw triples are triples whose elements have not yet been mapped into the underlying ontology.

Considering the example of the question *"Which volcanoes in Japan erupted since 2000?"*, Fig. 4 shows the table of the most important features annotated by spaCy, of which we note that the root node of the question is represented by the token *"erupted"*. From the generated subgraphs we observe that to the left of the root node there is the following syntactic dependency: nsubj("erupted","vulcanos") and to the right the following two: prep("erupted","since") and punct("erupted","?"). Therefore, to construct the first triple, onIQ traverses the left subgraph, constructing the following raw triple: `<?vulcanos "erupted" ?erupted>`.

Next the parser moves the target head to the nominal subject *"volcanos"* looking to the right and left for adjectives and prepositions to use when constructing the next raw triple. To the left of the new head, one identifies the relationship det("vulcanos","Which") and on the right prep("vulcanos","in"), which implies the building of the next raw triple: `<?vulcanos "location" Japan>`. Finally, the target head is moved to the preposition from which the last raw triple was constructed, but since the subgraphs generated for this token are empty, the cycle of constructing raw triples will end.

Comparing the two raw triples generated we can notice that in the former case, the names of the three components are derived from the question tokens, whereas in the latter case, only the subject and the object are derived from them, while the property gets the alias *"location"*. This alias is determined by analysing the preposition and the type of resource it refers to (a country).

B. Lexicon

Building triples involves applying the information extraction techniques described in section *IV. Information Extraction* to the raw triples prepared in the previous step. Basically, first the resource from which the searched instance, dbo:Volcano, is derived is identified and the first triple `<?volcanos rdf:type dbo:Volcano>` is constructed. Next, given that the raw triple `<?vulcanos "location" Japan>` contains the named entity *"Japan"*, Spotlight will be used to determine the DBpedia resource that describes this named entity, and then will identify the property that best fits this context to generate the triple `<?vulcanos dbo:locatedInArea dbr:Japan>`.

Finally, the system will also determine the property that best matches the extended context *"volcanoes in Japan erupted"*, thus generating the last triple: `<?volcanos dbo:eruptionYear ?erupted>`.

Token	Governor	Root Node	Lefts	Rights	Lemma	POS	TAG	DEP
Which	volcanos	False	[]	[]	which	DET	WDT	det
volcanos	erupted	False	[Which]	[in]	volcano	NOUN	NNS	nsubj
in	volcanos	False	[]	[Japan]	in	ADP	IN	prep
Japan	in	False	[]	[]	Japan	PROPN	NNP	pobj
erupted	erupted	True	[volcanos]	[since, ?]	erupt	VERB	VBD	ROOT
since	erupted	False	[]	[2000]	since	SCONJ	IN	prep
2000	since	False	[]	[]	2000	NUM	CD	pobj
?	erupted	False	[]	[]	?	PUNCT	.	punct

Fig. 4. NLP features generated for the question *"Which volcanoes in Japan erupted since 2000?"*.

To increase the accuracy of identifying the most suitable properties, the following rules have been established:

- dbo:locatedInArea property is used to determine where a subject derived from the dbo:NaturalPlace class is located.
- dbo:elevation property is used to determine the height of a subject derived from the dbo:Mountain class.
- dbo:birthDate property is used if the lemmatised form of the property for which a match is sought is *"old"* or *"young"*.
- dbo:populationTotal property is used to determine the total population living in a geographical location.
- foaf:nick property is used to determine the nickname of a person.

The advantage of using a specific set of rules to improve results is clear: it is specific to stored triples. For example, in the case of DBpedia resources derived from the dbo:Mountain class, which describe real-world mountains, it is natural that the reference to their height is made to the word *"elevation"*. On the other hand, the disadvantage of using such an approach is that these rules, being specific, cannot be applied to every triplestore, but must be adjusted.

C. SPARQL Query Building

After identifying triples, it is necessary to determine the filtering rules, for which process, onIQ seeks to identify adverbs, adpositions and subordinating conjunctions by which temporal expressions are related to the subjects and objects of the triples. Then, by traversing the dependency tree, the temporal expression is also identified, and finally, based on a set of heuristic rules, the filter statement is constructed.

Questions starting with *"how many"* or *"how much"* are usually questions for which the expected answer is an aggregation, therefore it is necessary to use the aggregation function COUNT, and if the questions contain superlative adjectives, it is a clear indication that the result should be both ordered using the ORDER BY statement and the LIMIT 1 solution modifier applied in order to limit the result to the most positive or negative result. The determination of the sort direction (ascending or descending) is related to the positivity of the adjectives. Thus, if the most positive result is to be identified, the order modifier DESC must be applied, and if the most negative result is to be identified, the order modifier ASC must be applied.

In the above example, *"since"* is the subordinating conjunction which is in the following dependent relationship: prep("erupted","since") . Therefore, after traversing the dependency tree, the token *"2000"* will be associated with calendar years, and then the following filter rule will be constructed: `FILTER(year(?erupted) >= 2000)`. Putting all the steps together, the final query takes the shape shown in Listing 3.

VI. Evaluation and Discussion

The evaluation was performed on the QALD-8 test dataset which contains 42 heterogeneous questions for which SPARQL queries can be constructed with respect to the provided RDF data [23], being focused only on questions expressed in English. The measurement of the results is materialized through the FAIR benchmarking platform GERBIL QA, an open source tool for automated evaluation of QA systems [24] that allows users to run tests just by selecting the target corpus and uploading the answers generated by the developed QA system in the agreed format. Basically, to measure the results, GERBIL QA uses Precision, Recall and F-measure parameters.

```
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbr: <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-
ns#>

SELECT DISTINCT ?volcanos
WHERE {
  ?volcanos  dbo:eruptionYear     ?erupted .
  ?volcanos  dbo:locatedInArea    dbr:Japan .
  ?volcanos  rdf:type             dbo:Volcano .
  FILTER (year(?erupted) >= 2000)
}
```

Listing 3. SPARQL query generated for the question *"Which volcanoes in Japan erupted since 2000?"*.

The following rules have also been adopted:

- if for a question *q* golden answerset is empty and the evaluated system generates an empty answer, the three parameters are assigned the value 1.
- if for a question *q* golden answerset is empty but the evaluated system generates any answerset, the three parameters will be assigned the value 0.
- if for a question *q* there is a golden answerset but the evaluated system provides an empty answerset, it is assumed that the system was not able to answer and the three parameters are assigned the value 0.

Table 2 gives an overview of the results of the experiments performed against the QALD-8 dataset for the selected QA systems. The onIQ evaluation was performed with GERBIL version 0.2.3, using both the configured version with the set of rules mentioned in subsection *V.B. Lexicon* and the unconfigured version.

TABLE II. Results obtained by onIQ and other state-of-the-art QA systems

QA System	Precision	Recall	F-measure
gAnswer2 [8]	0,337	0,354	0,345
onIQ [configured] [25]	**0,478**	0,370	0,417
onIQ [unconfigured] [26]	0,287	0,326	0,305
QANARY [27]	0,240	0,230	0,235
QAnswer [9]	0,452	0,480	0,466
qaSQP [10]	0,459	0,463	0,461
TeBaQA [11]	0,476	**0,488**	**0,482**
WDAqua-core0 [9]	0,391	0,407	0,399

It can be noticed that after configuring the framework, there were reported the highest values where various numbers have been reported in the literature, achieving the best results in terms of Precision but not in terms of Recall. This means that although the number of questions for which valid SPARQL queries could be built is smaller, the correctness of the answers is higher than what other systems report. On the other hand, in the case of the unconfigured scenario, an

important decrease of the Precision has been reported in parallel with negligible differences in Recall.

Basically, we can observe that onIQ gives better results on a limited set of questions than the systems analysed but only after applying a set of well-established rules. Although the testing was performed on questions against DBpedia, the system can be extended to support any other triplestore, the only condition being the development of its own NER mechanism. Next, the direction is to improve syntactic parsing to increase the complexity of the queries on which it can be applied and to develop a default NER mechanism so that this task no longer falls on the users.

Moving forward, the aim is to enhance the syntactic parsing capabilities, enabling the system to handle more complex queries. Additionally, there's a plan to develop a NER mechanism to facilitate the use of the proposed NLI against any triplestore and perform new tests to validate the framework against datasets represented according to Europeana Data Model (EDM).

DISCLOSURE

This paper is based on material from the author's PhD thesis entitled *"Open Data Services Using The Semantic Web"*, to be published by *National University of Science and Technology Politehnica Bucharest*, 2026.

REFERENCES

[1] J. Rusher, "Rhetorical device: Triple Store". [Online]. https://www.w3.org/2001/sw/Europe/events/20031113-storage/positions/rusher.html, last accessed: May 2026.

[2] M. Dubey, S. Dasgupta, A. Sharma, K. Höffner, and J. Lehmann, "AskNow: A Framework for Natural Language Query Formalization in SPARQL", European Semantic Web Conference (ESWC 2016), LNCS, vol. 9678, pp. 300-316, Springer, Cham, 2016. doi:10.1007/978-3-319-34129-3_19.

[3] J. Berant, A. Chou, R. Frostig, and P, "Semantic Parsing on Freebase from Question-Answer Pairs". Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, pp. 1533–1544, 2013.

[4] D. Banerjee, P. B. Nair, J. N. Kaur, R. Usbeck, and C. Biemann, "Modern baselines for SPARQL Semantic parsing". Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2022. doi:10.1145/3477495.3531841.

[5] QALD-8 test dataset. [Online]. https://github.com/ag-sc/QALD/blob/master/8/data/qald-8-test-multilingual.json, last accessed: May 2026.

[6] I. C. Dorobăț, and V. Posea, V., "onIQ: an ontology-independent natural language interface for building SPARQL queries", International Conference on Intelligent Computer Communication and Processing (ICCP 2020), pp. 139–144, Cluj-Napoca, Romania. doi:10.1109/ICCP51029.2020.9266272.

[7] GERBIL QA. [Online]. http://gerbil-qa.aksw.org/gerbil/, last accessed: May 2026.

[8] L. Zou, R. Huang, H. Wang, J. X. Yu, W. He, and D. Zhao, "Natural language question answering over RDF: a graph data driven approach", International Conference on Management of Data (SIGMOD 2014), pp. 313–324, 2014, Snowbird, UT, USA. doi:10.1145/2588555.2610525.

[9] D. Diefenbach, A. Both, K. D. Singh, and P. Maret, "Towards a question answering system over the Semantic Web", Semantic Web, vol. 11(3), pp. 421–439, 2020. doi:10.3233/SW-190343.

[10] W. Zheng, and M. Zhang, "Question Answering over Knowledge Graphs via Structural Query Patterns", arXiv (Cornell University), 2019. doi:10.48550/arXiv.1910.09760.

[11] D. Vollmers, R. Jalota, D. Moussallem, H. Topiwala, A. N. Ngomo, and R. Usbeck, "Knowledge Graph question answering using Graph-Pattern isomorphism", Studies on the semantic web, 2021. doi:10.3233/SSW210038.

[12] spaCy Homepage. [Online]. https://spacy.io/, last accessed: May 2026.

[13] A. Rodriguez, "Part-of-speech tagging", Wiley Interdisciplinary Reviews: Computational Statistics, vol. 4(1), pp. 107–113, 2011. doi:10.1002/wics.195.

[14] R. Weischedel, et al., "OntoNotes Release 5.0". [Online]. https://catalog.ldc.upenn.edu/LDC2013T19, last accessed: May 2023.

[15] N. Colic, and F. Rinaldi, "Improving spaCy dependency annotation and PoS tagging web service using independent NER services", Genomics & Informatics, vol. 17(2), e21, 2019. doi:10.5808/GI.2019.17.2.e21.

[16] R. G. Athreya, S. K. Bansal, A.-C. N. Ngomo, and R. Usbeck, "Template-based Question Answering using Recursive Neural Networks", IEEE 15th International Conference on Semantic Computing, pp. 195–198, 2021. doi:10.1109/ICSC50631.2021.00041.

[17] M. Honnibal, and M. Johnson, "An Improved Non-monotonic Transition System for Dependency Parsing", Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, pp. 1373–1378, 2015. doi:10.18653/v1/D15-1162.

[18] J. Nivre, and J. Nilsson, "Pseudo-Projective Dependency Parsing", Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics, pp. 99–106, 2005. doi:10.3115/1219840.1219853.

[19] K. B. Hall, and V. Novák, "Corrective dependency parsing", Springer eBooks, pp. 151–167, 2010. doi:10.1007/978-90-481-9352-3_9.

[20] M. Morsey, J. Lehmann, S. Auer, C. Stadler, and S. Hellmann, "DBpedia and the live extraction of structured data from Wikipedia", Program: Electronic Library and Information Systems, vol. 46(2), pp. 157–181, 2012. doi:10.1108/00330331211221828.

[21] P. N. Mendes, M. Jakob, A. García-Silva, and C. Bizer, "DBpedia Spotlight: Shedding Light on the Web of Documents", Proceedings of the 7th International Conference on Semantic Systems, pp. 1–8, 2011. doi:10.1145/2063518.2063519.

[22] M. Chabchoub, M. Gagnon, and A. Zouaq, "FICLONE: Improving DBpedia Spotlight Using Named Entity Recognition and Collective Disambiguation", Open Journal of Semantic Web, vol. 5(1), pp. 12–28, 2018.

[23] R. Usbeck, A.-C. N. Ngomo, F. Conrads, F. Röder, and G. Napolitano, "8th Challenge on Question Answering over Linked Data (QALD-8)", Semdeep/NLIWoD@ISWC, 2018.

[24] R. Usbeck, M. Röder, M. M. Hoffmann, F. Conrads, J. Huthmann, A. Ngonga-Ngomo, C. Demmler, and C. Unger, "Benchmarking question answering systems", Semantic Web, vol. 10(2), pp. 293–304, 2019. doi:10.3233/SW-180312.

[25] GERBIL Experiment – Configured onIQ. [Online] http://gerbil-qa.cs.uni-paderborn.de:8080/gerbil/experiment?id=202307050036, last accessed: May 2026.

[26] GERBIL Experiment – Unconfigured onIQ. [Online]. http://gerbil-qa.cs.uni-paderborn.de:8080/gerbil/experiment?id=202309090002, last accessed: May 2026.

[27] R. Jalota, D. Vollmers, D. Moussallem, & A.-C. N. Ngomo, "LAUREN - Knowledge Graph Summarization for Question Answering", IEEE 15th International Conference on Semantic Computing, pp. 221–226, 2021. doi:10.1109/ICSC50631.2021.00047.